

AlphaStack: AI-powered project generator with Multi-Agent Systems

HyperKuvid-Labs

Submitted to ICML 2026

Abstract

AlphaStack is a novel AI-powered project generator that transforms natural language descriptions into complete, production-ready codebases with Docker configurations and automated testing. By utilizing a multi-agent architecture with iterative self-healing and comprehensive validation across diverse programming paradigms, AlphaStack tackles the complex problem of automated, end-to-end software synthesis. This paper outlines our intelligent multi-agent framework involving a Planning Agent and Correction Agent, comprehensive docker-based validation, and results from an extensive evaluation framework across modern programming languages.

1 Introduction

Modern AI code assistants focus on localized snippet generation or context-limited autocompletion. AlphaStack introduces an intelligent end-to-end multi-agent framework that parses a software prompt into a comprehensive multi-file structure, manages dependency resolution, and automatically provisions isolated environments using Docker. When builds or tests fail, AlphaStack exhibits self-healing behavior through its iterative tool-augmented reasoning. The pipeline is designed for robustness and can generate projects ranging from simple REST APIs to complex CUDA kernels.

2 Methodology

The generation pipeline within AlphaStack consists of seven sequential phases. It starts with the Software Blueprint Generation, progressing through File Generation, Dockerfile Generation, Dependency Analysis, Dependency File Generation, and Dependency Resolution. The final phase is the Docker Testing Pipeline, which activates the AI Planner Agent Loop. This loop iteratively compiles, tests, and refines the codebase. The multi-agent system comprises:

- **Planning Agent:** Analyzes build and test errors, and generates comprehensive fix strategies utilizing tools like file reading, dependency tracking, and command execution.
- **Correction Agent:** Executes the plans securely, incorporating language-specific parsers to prevent syntax regressions.

These agents operate in a sandboxed Docker environment to ensure security and resource isolation.

3 Architecture Diagram

AlphaStack’s holistic process from prompt to a validated repository is illustrated in the architectural diagram below (Figure 1).

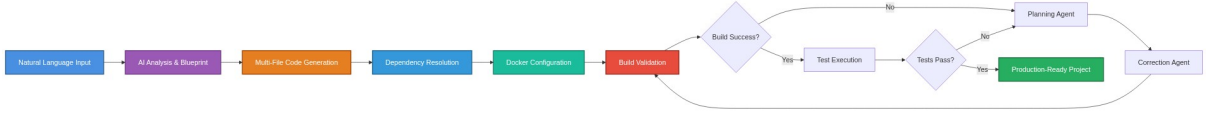


Figure 1: AlphaStack generation and multi-agent validation pipeline.

4 Results

AlphaStack is rigorously evaluated on 40 challenges across four languages (CUDA, Go, Rust, TypeScript), organized into a 4-Tier difficulty system. In our preliminary analysis using different state-of-the-art base language models on the HumanEval and MDDP benchmark suites, we observed promising results. Figure 2 presents the tentative performance metrics for `gpt-5.2`, `glm-5`, `minimax2.5`, and `claude sonnet 4.6`.

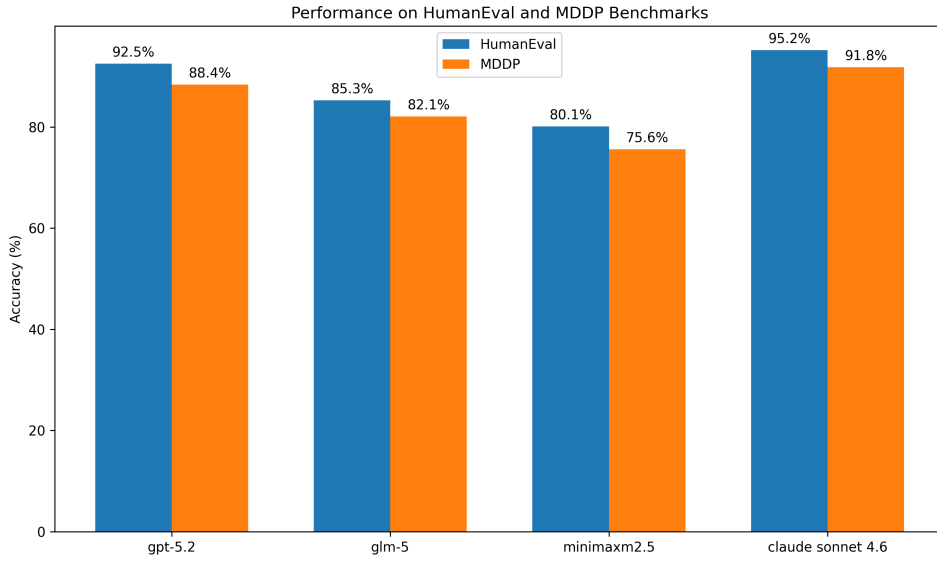


Figure 2: Performance on HumanEval and MDDP Benchmarks by Model.

5 Conclusion

AlphaStack demonstrates the efficacy of a multi-agent, iteratively self-healing architecture in the realm of automated code generation. By isolating environments via Docker, maintaining detailed context structures, and implementing dedicated Planner and Corrector agents, the system reliably synthesizes production-ready software. Future work will expand the evaluation suite and explore integrating more granular test-driven development methodologies during the generation phase.

Supplementary Material

Additional artifacts, the 40-challenge evaluation suite, and full tool schematics are available in the open-source repository at <https://github.com/HyperKuvId-Labs/alpha-stack>.